

# Weishu Zhang

626-566-4310 | [weszhang@berkeley.edu](mailto:weszhang@berkeley.edu) | [LinkedIn](#) | [GitHub](#) | [Website](#)

U.S. work authorized; available December 2026. Backend-leaning full-stack engineer building production agent systems and enterprise workflow runtimes.

## EDUCATION

### University of California, Berkeley

Berkeley, CA

*Bachelor of Science in Electrical Engineering and Computer Sciences*

*May 2027*

– **GPA: 4.0/4.0**

– **Relevant Coursework:** Efficient Algorithms, Network Science, Artificial Intelligence

## TECHNICAL SKILLS

**Languages:** Python, TypeScript/JavaScript, SQL, Java, Go, C/C++

**Full Stack:** React, Next.js, Node.js, FastAPI, PostgreSQL, Redis, SQLite, REST APIs

**AI & Agents:** LangGraph, MCP, LLM APIs, agent runtimes, structured generation, PyTorch, vLLM, QLoRA

**Infrastructure:** Docker, Kubernetes, pytest, distributed jobs, workflow orchestration, observability, CI/CD

## EXPERIENCE

### Amazon

*Incoming Software Development Engineer Intern*

*Sep 2026 – Nov 2026*

### ASML

San Jose, CA

*Software Engineering Intern, Agent Runtime & Test Infrastructure*

*2026 – Present*

- Built an ontology-driven **runtime agent infrastructure** that models enterprise entities, relationships, permissions, and actions as a shared semantic layer, enabling agents to discover governed context and execute workflows consistently across internal systems.
- Developed a **Test Plan Runtime Agent** and workspace harness for stateful workflows: synthesized EPDS and Jira documentation to create or enhance TPS artifacts, proposed test-task improvements and execution plans for human approval, then asynchronously managed task state and reconciled results back into the TPS.
- Designed **token-efficient, reliable, and reusable** agent primitives using scoped context retrieval, typed tool contracts, approval gates, idempotent writes, resumable execution, distributed job claiming, and failure recovery; reduced repetitive QA data entry by over **80%** and automated approximately **90%** of the review workflow.

### Open Ludus

Remote

*Co-Founder & Software Engineer, Agent Systems*

*2026 – Present*

- Built an **AI-native full-stack work surface** in TypeScript and Python where agents generate declarative interfaces, users review and edit live application state, and approved actions are committed to external systems.
- Designed a streaming frontend runtime that compiles structured agent output into interactive forms, approval queues, progress views, and intervention controls within ongoing chat sessions.
- Integrated LangGraph and MCP-based agents with typed frontend events, persistent workflow state, human approval gates, and idempotent commit adapters for reliable long-running operations across CRM and productivity tools.

### Berkeley Artificial Intelligence Research (BAIR)

Berkeley, CA

*Undergraduate Research Assistant*

*Jan 2026 – Present*

- Led expert task acquisition for the **Agent Human Last Exam** benchmark, recruiting **30+ domain experts** and contributing to the collection of **80+** complex, cross-domain agent tasks.
- Built an LLM-powered proposal agent that converts rough expert ideas into structured benchmark tasks, supporting iterative refinement, constraint checking, and higher-quality submissions.

### University of California, San Francisco

San Francisco, CA

*Software Research Assistant*

*Aug 2025 – Present*

- Built a production-oriented long-context clinical information extraction pipeline, increasing inference throughput by **3x** and reducing VRAM usage by approximately **80%** using vLLM, paged attention, quantization, and batched execution.
- Created a **4,000+**-example domain dataset, fine-tuned Qwen models with QLoRA, and implemented validation and self-consistency checks that maintained **100% schema-valid** structured outputs.

## OPEN SOURCE

**PyTorch** | [torch.compile](#), [torch.export](#), [Dynamo PRs](#)

- Contributed production fixes to **pytorch/pytorch** across compiler and export internals, resolving eager-versus-compiled correctness gaps, tracing failures, and export regressions with targeted tests and maintainer-reviewed patches.

## SELECTED PROJECTS

**GradeView** | [React](#), [TypeScript](#), [Python](#), [PostgreSQL](#), [Redis](#) [Live](#) | [Demo](#) | [Poster](#)

- Refactored a production learning-analytics platform used in Berkeley courses into containerized React, Node.js, and Python services; standardized PostgreSQL as the source of truth and added Redis cache prewarming to improve reliability and read performance.